

Portfolio Reflection

Jacob Perez

2025-06-08

Download the source code for the template here:

[reflection-template.qmd](#)

Independent Learning (IL):

These objectives show your ability to seek out new information and adapt to new tools to solve data analysis problems.

[IL-1] Adding new skills:

- I can find and adopt new packages to accomplish tasks.
- I can adapt to different syntax styles (tidy, base, formula style, data.table).

Level: 2

Justification:

```
mpg %>%  
  mutate(  
    trans_type = str_split_i(trans, "\\(", i = -1),  
    trans_type = str_remove(trans_type, "\\)"),  
    trans = str_split_i(trans, "\\(", i = 1)  
  ) %>%  
  rename(trans_style = trans)
```

In Lab 1 this shows how I can use `str_splits` and `mutate` to accomplish data cleaning. These functions come from the `dplyr()` package which also shows the ability to adopt new packages to accomplish tasks.

[IL-2] Online resources:

- I can use online resources (e.g., Google, ChatGPT, StackOverflow, YouTube) to solve problems, debug, or find new tools.
- I can use tutorials, etc. to enhance my understanding of new concepts.
- I can find source code for similar projects to use as starting points for my own.

Level: 2

Justification:

```
library(leaflet)
library(sf)
library(rnaturalearth)
library(dplyr)

world_sf <- ne_countries(scale = "medium", returnclass = "sf")

country_names <- tibble::tibble(
  country_name = c("Bosnia and Herzegovina", "Czech Republic",
                  "Dominican Republic", "Eswatini", "Ivory Coast",
                  "Macedonia", "Northern Cyprus", "Palestinian Territories",
                  "Republic of Congo", "The Gambia", "United States"),
  name = c("Bosnia and Herz.", "Czechia", "Dominican Rep.", "eSwatini",
           "Côte d'Ivoire", "North Macedonia", "N. Cyprus", "Palestine",
           "Congo", "Gambia", "United States of America")
)

leaflet_df <- leaflet_df %>%
  left_join(country_names, by = "country_name") %>%
  mutate(name = coalesce(name, country_name))

leaflet_map_df <- left_join(world_sf, leaflet_df, by = "name")

lf_labels <- sprintf(
  "<strong>%s</strong><br/>%s<br/>Interest: %g%%",
```

```

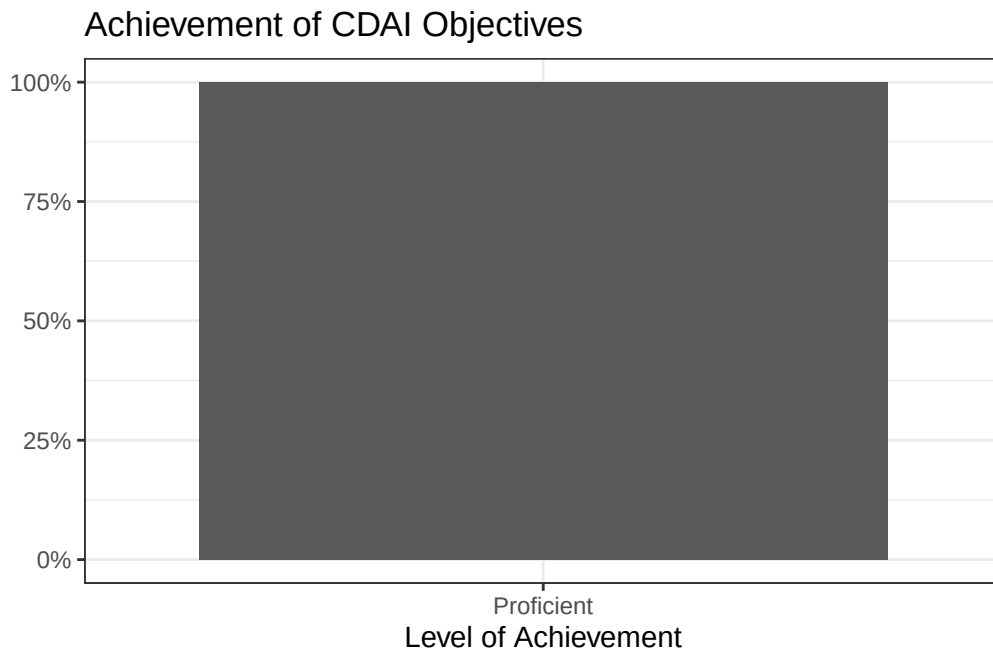
leaflet_map_df$name,
leaflet_map_df$region,
round(leaflet_map_df$prop * 100, 1)
) %>% lapply(htmltools::HTML)

pal <- colorNumeric(
  palette = "YlGnBu",
  domain = leaflet_df$prop
)

```

This was an example pulled from lab 2 where we had to create a leaflet plot. To correctly create the labels for each country and make sure they correctly lined up with the dataset I used ChatGPT which shows my ability to use online resources to accomplish tasks/debug. Without being able to efficiently use ChatGPT I would not have been able to correctly match country labels to the countries.

IL Summary



Reproducible Workflow (RW):

These objectives show your ability to produce artifacts and deliverables that are organized, documented, version tracked, and responsibly designed.

[RW-1] File, code, and data management:

- I can use Git and GitHub to track my progress (creating repos, cloning, forking, pull requesting).
- I always use R Projects and the {here} package to organize my scripts, notebooks, data, and applications.
- I always use pull requests when collaborating with others.

Level: 1

Justification:

```
library(readxl)

wgm2018 <- read_excel(here::here("wgm2018-dataset-crosstabs-all-countries.xlsx"),
                      sheet = 2)

wgm20183 <- read_excel(here::here("wgm2018-dataset-crosstabs-all-countries.xlsx"),
                      sheet = 3)
```

This is an example from lab 2 again about reading in the data from one of my scripts. To create reproducible code I used the `here()` command from the `here()` package to read in the data correctly. This chunk shows me using the `here()` package to better organize my scripts.

[RW-2] Notebooks:

- I can use Quarto to produce a reproducible notebook and polished rendered documents
- I can use appropriate chunk options (`echo`, `error`, `cache`, etc.) to render my Quarto document quickly and cleanly.

Level: 2

Justification:

```
title: "Lab 2"
subtitle: "Advanced Data Visualization"
editor: source
embed-resources: true
execute:
  warning: false
```

This is again from lab 2 but shows the yaml header from this lab. I was able to use the `warning: false` command in the yaml header so that none of the warnings from the code would show when the file rendered. This has become a useful tool especially when moving the files to my portfolio as you don't want to see warning messages pop up in the section for blog posts.

[RW-3] Code style

- My code is clear, readable, well-organized, and well-commented.

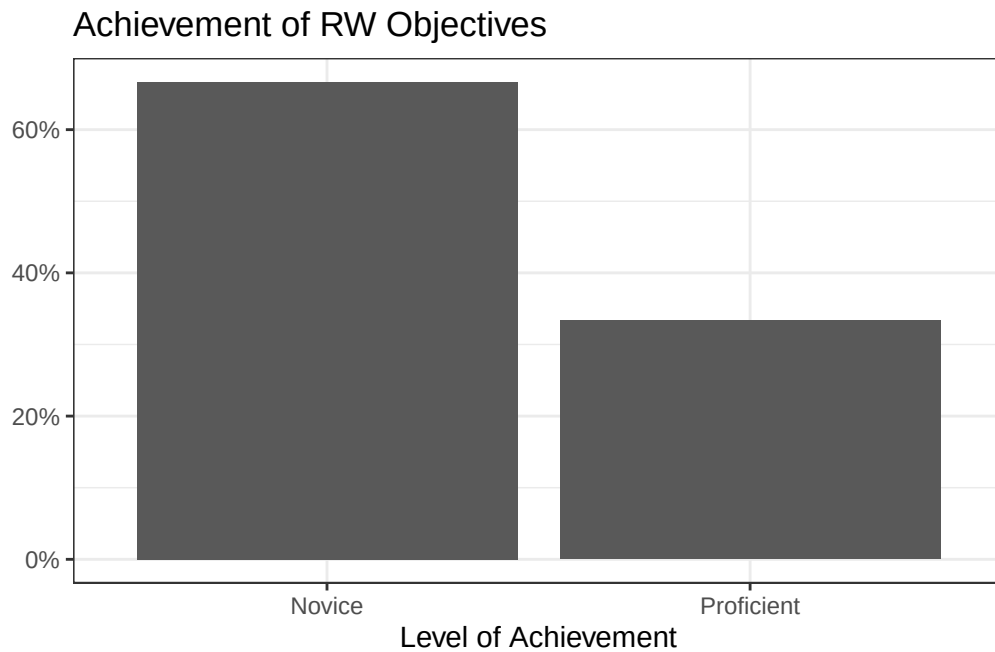
Level: 1

Justification

```
2 + a
```

I give myself a 1 here because I think I have a lot to improve in terms of commenting and nicely formatting my code. I do use code chunk labels which helps clarify and keep the R script neat, but I could improve by commenting more to help followers with my code.

RW Summary



Technical Communication (TC):

These objectives show your ability to communicate the processes you have implemented in your code, as well as the data conclusions and results.

[TC-1] Project summaries:

- I can create clear and succinct summaries of a project.
- I accurately interpret statistical or modeling results.
- I consider the appropriate scope and impact of my project results.

Level: 1

Justification:

This shows my quarto dashboard where I created a clear and succinct summary of the goal of the project. This created a story that shows how trust in science changed from the year 2018

to 2020 on a regional level and country level. I gave myself a 1 because I know that there is more room for improving statistical or modeling results.

[TC-2] Documentation:

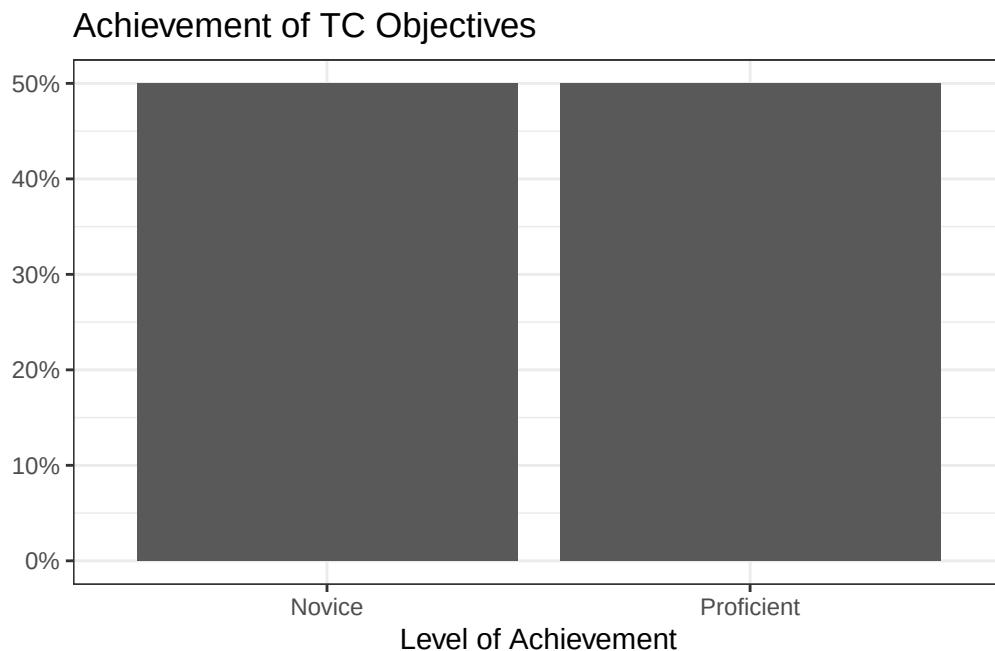
- I can create a user-friendly dashboard.
- I provide ample documentation for my custom functions.

Level: 2

Justification:

The justification for my 2 is from above from my quarto dashboard. I really enjoyed the story that my dashboard tells as well as the layout/interactivity. I feel like it tells a story but allows for user interaction so they can also ask and answer questions that they have themselves.

TC Summary



Data Manipulation (DM):

These objectives relate to the collection, cleaning, processing, and preparing of datasets for analysis.

[DM-1] Data Preparation

- I can read in datasets to R, including untidy ones.
- I can clean datasets to deal with missing data, typos, poor formatting, etc.

Level: 1

Justification:

```
wgm2018_joined <- left_join(wgm2018,
                             df_countries,
                             by = c("WP5" = "country_id")) |>
mutate(region = case_when(
  Regions_Report %in% c(1, 2, 4, 5) ~ "Africa",
  Regions_Report %in% c(6, 7, 8) ~ "Americas",
  Regions_Report %in% c(9, 10, 11, 12, 18) ~ "Asia",
  Regions_Report %in% c(3, 13) ~ "Middle East and North Africa",
  Regions_Report %in% c(14) ~ "Former Soviet Union",
  Regions_Report %in% c(15, 16, 17, 0) ~ "Europe"
)) |>
select(country_name, region, wgt, PROJWT, Q25, Q9, Q8, Q6)
```

This shows just some of my data collection process for the wgm dataset. In lab 2 we had to read in an excel file and clean a lot of messy data. This code shows how I can use `case_when()` to correctly rename the regions column and join two datasets, one with countries and their corresponding regions. I gave myself a 1 because I do not think I am the most efficient data cleaner and I feel like my data cleaning process is often messy and long.

[DM-2] Data Wrangling

- I can cleverly use pivoting, separating, grouping, and joining to wrangle data.
- I can use mapping (`purrr`) to perform repeated tasks.

Level: 1

Justification:

```
summary_df <- wgm2018_joined |>
  filter(!is.na(Q25)) |>
  group_by(country_name, region) |>
  summarise(
    total_responses = n(),
    count = sum(Q25 %in% c(1, 2)),
    prop = count / total_responses
  ) |>
  select(country_name, region, prop) %>%
  mutate(country_name = if_else(country_name == "Northern Cyprus,",
                                "Northern Cyprus",
                                country_name))
```

Again I stole this from lab 2. I believe that this does a great job at showing how to use `group_by()`s and `filter()`s really well. There are some `select()` statements and `mutate()` statements as well, which all show off dplyr commands. Again I gave myself a 1 because I believe I can do this at a very basic level and I still need to look up the functions sometimes.

[DM-3] Data Formats

- I can use API urls to access JSON data and convert it into a data frame
- I can scrape data from the web and convert it into a data frame

```
## Fill in the line below with your self-assigned level for this objective.
my_level <- 2

portfolio_levels <- portfolio_levels %>%
  bind_rows(tibble(Objective = "DM-3",
                    Level = my_level))
```

Level: 2

Justification

```

get_json_info <- function(df){
  #make empty list
  results <- list()

  #iterate through the df to get each state, lat, long, and plug each into url.
  for(i in 1:nrow(df)){
    state <- df$state[i]
    latitude <- df$latitudes[i]
    longitude <- df$longitudes[i]
    url <- paste0("https://api.g7vrd.co.uk/v1/satellite-passes/25544/", latitude, "/", longitude)

    # make an API request for each url
    response <- GET(url)

    #if the response works
    if (http_status(response)$category == "Success") {

      # grab the json content, the first 3 satellite passes
      json_data <- fromJSON(rawToChar(response$content))
      passes_data <- head(json_data$passes$tca,3)

      # Store the relevant data in the list
      results[[i]] <- list(state = state, latitude = latitude, longitude = longitude, passes = passes_data)
    }

    #if the response didn't work
    else {
      cat("Error fetching data for latitude:", latitude, ", longitude:", longitude)

      results[[i]] <- list(state = state, latitude = latitude, longitude = longitude, error = "Error")
    }
  }
  #after the looping, return the list
  return(results)
}

api_data <- get_json_info(state_capitals_clean)

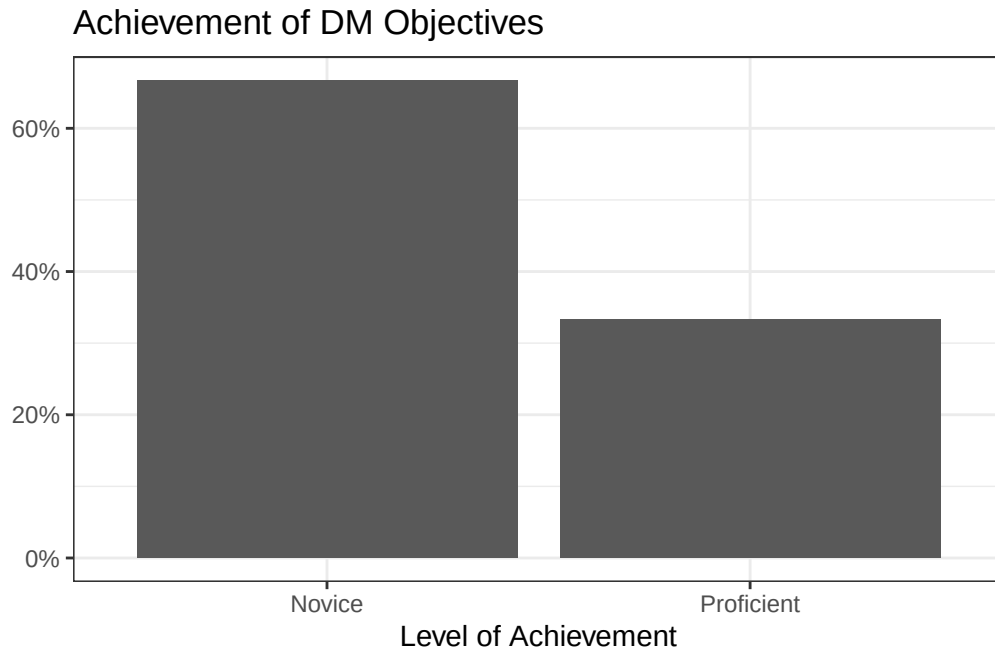
# used Gemini to help with the above function

```

This function shows an example of when my partner and I used apis from the satellite website

to scrape data on passtimes. This gave us good practice at using api functions like `fromJSON()` and `http_status()` to scrape for API data. It took some practice at first to understand just what the url was scraping and how we needed to scrape over multiple times. Originally we were scraping passtimes for just one city at a time. This was not the goal of the lab so we had to iterate over the whole dataframe (or all capitals) in the function using our API scrape functions.

DM Summary



Professional Visualization (PV):

[PV-1] Clear & Accessible Visualizations

- I can make my plots more clear by removing the legend and adding annotations.
- I can edit the titles, subtitles, captions, axis labels, etc. to create a clearly labelled plot.
- I can choose colors (“scales”) and themes to make a visually pleasing and accessible plot.

Level: 2

Justification:

```
ggplot(trust2018, aes(x = percent)) +  
  geom_density(binwidth = 3, fill = "#65a345", color = "white") +  
  labs(  
    title = "Distribution of Percentages",  
    x = "% of Responses",  
    y = NULL  
  ) +  
  theme_minimal() +  
  theme(  
    panel.grid.major = element_blank(),  
    panel.grid.minor = element_blank()  
  )
```

Although this is not my most beautiful or complex graph I really enjoy the layout and aesthetic and I believe it uses a lot of the tools we learned in a very simple way to create a clean but dynamic graph. This code was pulled from lab3 and can be seen as the graphs in the static dashboard blog post. These graphs use a minimal theme, labs, and the theme() function to title, simplify, and remove unnecessary labels that are otherwise explained in the static dashboard. I also really enjoy the green color on this graph.

[PV-2] Dynamic Visualizations

- I can use a package like {gganimate} to create self-contained gifs.
- I can use a package like {plotly}, {ggplotly}, {leaflet}, {ggirafe}, etc. to make interactive html widgets.

Level: 2

Justification:

```
p <- ggplot(summary_df, aes(x = prop, y = country_name, color = region)) +  
  geom_point() +  
  geom_vline(data = region_medians,  
            aes(xintercept = median_prop),  
            color = "black",
```

```

        linetype = "twodash",
        size = 0.3,
        alpha = 0.8) +
geom_text(data = region_medians,
  aes(x = median_prop,
      y = Inf,
      label = paste0("Median: ", round(median_prop * 100, 1), "%")),
  color = "black",
  size = 3,
  fontface = "bold") +
labs(
  x = "% of people who believe vaccines are safe",
  y = NULL,
  title = "Belief in Vaccine Safety - {closest_state}"
) +
theme_minimal(base_size = 12) +
theme(
  plot.margin = margin(4, 60, 4, 4),
  legend.position = "none",
  axis.text.y = element_blank()
) +
geom_text(data = labels,
  aes(label = country_name),
  hjust = 0,
  size = 3,
  color = "black",
  nudge_x = 0.015) +
coord_cartesian(clip = "off") +
scale_color_brewer(palette = "Set2") +
transition_states(region,
  transition_length = 2,
  state_length = 2) +
ease_aes('cubic-in-out')

anim <- animate(p, renderer = gifski_renderer())
anim_save("vaccine.gif", animation = anim)

knitr::include_graphics("vaccine.gif")

```

This graph is from my lab2 and on my blog post under improving visualizations. This shows the first animated plot that shows up when you click on the corresponding blog post. This shows a gganimate plot that even has a changing title based on the transition state which

corresponds to the correct region. I think the visualization could look nicer on the eye which is why I don't believe I deserve a 3, but I think as a basic animated plot, it tells a nice story and uses some neat tricks with a changing name based on the closest state.

[PV-3] Interactive Visualizations

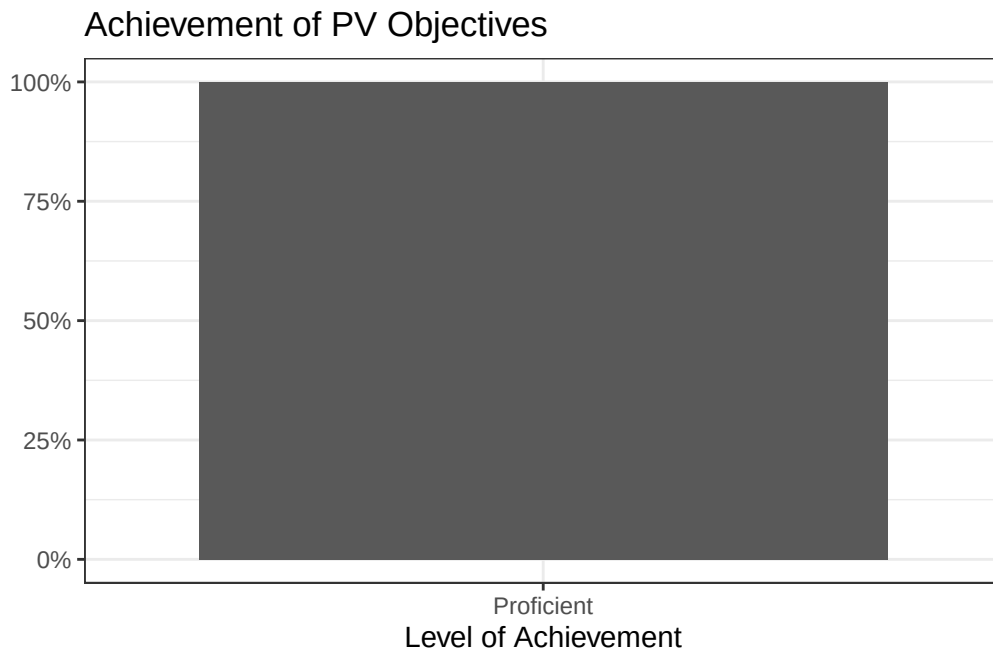
- I can use Shiny or webR to create visualizations that react to a user's input.

Level: 2

Justification:

This is from lab4 and pulled from my blog post on Shiny apps and creating an interactive quarto dashboard. This link shows the Shiny dashboard that again tells a similar story to my static dashboard. I love how both tell a story without the interactive elements and the interactive elements enhance the user experience by allowing them to customize what is being told based on what questions they want to ask. I didn't give myself a 3 because again I think the overall design can be improved but I still think for the interaction and story telling it deserves a 1.

PV Summary



Code Design, Algorithms, Iteration (CDAI):

These objectives ask you to design code-based approaches to statistical computing problems, usually involving iteration to a stopping condition.

[CDAI-1] R Programming Language

- I understand non-standard evaluation (aka “tidy eval” or “unquoted objects”), and I can use tunneling in my functions.
- I understand functional programming, and I can use functions as objects in my code design.

Level: 2

Justification:

```
library(dplyr)
library(rlang)

remove_outliers <- function(data, ..., sd_thresh = 3){
  vars <- enquos(...)

  filtered_data <- data

  for (var in vars) {
    var_name <- quo_name(var)

    if (!is.numeric(pull(filtered_data, !!var))) {
      warning(glue::glue("Variable '{var_name}' is not numeric."))
      next
    }

    var_mean <- mean(pull(filtered_data, !!var), na.rm = TRUE)
    var_sd <- sd(pull(filtered_data, !!var), na.rm = TRUE)

    filtered_data <- filtered_data %>%
      filter(abs(!!var - var_mean) / var_sd < sd_thresh)
  }
}
```

```

  return(filtered_data)
}

```

This function uses the `...` and `enquos()` to ensure that other specified parameters will be used later down in the function. This allows the user to specify as little or as many variables in the dataset and remove outliers for those variables.

[CDAI-2] Object Handling

- I have built in checks for possible input problems
- I can make reasonable choices in my code design about when to save intermediate objects.
- I can convert objects between types and structures as needed.

Level: 3

Justification:

```

library(dplyr)
library(rlang)

remove_outliers <- function(data, ..., sd_thresh = 3){
  vars <- enquos(...)

  filtered_data <- data

  for (var in vars) {
    var_name <- quo_name(var)

    if (!is.numeric(pull(filtered_data, !!var))) {
      warning(glue::glue("Variable '{var_name}' is not numeric."))
      next
    }

    var_mean <- mean(pull(filtered_data, !!var), na.rm = TRUE)
    var_sd <- sd(pull(filtered_data, !!var), na.rm = TRUE)

    filtered_data <- filtered_data %>%
      filter(abs(!!var - var_mean) / var_sd < sd_thresh)
  }
}

```



```
    return(filtered_data)
}
```

I wanted to use the same function since this function also shows how to meaningfully save intermediate objects and check for input object types. Like before we use the `...` and `enquos` for tunneling within our function. This comes with some problems as we have to check for input types since users are now able to input any type of variable they choose. To check for this we put a warning to check only for numeric variables based on the variable names the user input. To properly check for variables being numeric we also have to properly save the variables properly inside the function. This is done at the beginning of the function by using the `vars <- enquos(...)`, so now we can properly check the variable type or use other functions on the inputs by using `vars` later in the function.

[CDAI-3] Speed and Efficiency

- I can recognize moments of possible slowdown in my code, and use built-in functions or parallelizing to speed them up.
- I always use and design vectorized functions whenever possible.

Level: 2

Justification:

```
write.csv(trust2018, here::here("trust2018.csv"))
write.csv(trust2020, here::here("trust2020.csv"))
write.csv(trust_change, here::here("trustchange.csv"))
write.csv(df_wgm2018, here::here("wgm2018.csv"))
write.csv(df_wgm2020, here::here("wgm2020.csv"))
```

Yes, when developing the shiny app I first needed to process the data and make my cleaned datasets into csv files. This was used before my lab 4 as the shiny interactive dashboard would not render when loading in the original excel file that was huge. By preprocessing and cleaning the dataset into smaller csv files and then reading in those csv files I was able to successful render my Shiny app.

[CDAI-4] Supporting Functions

- I write helper / shortcut functions to streamline repeated tasks and make my code easier to read.
- I use intermediate functions to streamline repeated or looping processes.

Level: 2

Justification:

This is my same justification as from before, preprocessing and cleaning. This makes reading in your code look cleaner and the overall rendered file look clean. By throwing the pre-processing steps on another script this can really speed up your actual file you are working with as well as make the data reading process look a lot nicer.

[CDAI-5] Algorithmic Process

- I can invent and implement my own iterative algorithm.
- My loops are clean and efficient.
- I have built in checks for possible problems or extreme cases in the algorithm.

Level: 2

Justification:

```
# Define the base URL
base_url <- "https://www.cheese.com/alphabetical/"

# Function to scrape a single page
scrape_page <- function(page_number) {
  page_url <- paste0(base_url, "?page=", page_number)
  Sys.sleep(1)

  # Read the HTML content from the webpage
  webpage <- read_html(page_url)

  # Pull cheese names
  cheese_names <- webpage %>%
    html_elements("h3 a") %>%
```

```

    html_text()

# Pull cheese URLs
cheese_data <- webpage %>%
  html_elements("h3 a") %>%
  html_attr("href") %>%
  paste0("https://cheese.com", .)

# Pull cheese images
cheese_images <- webpage %>%
  html_elements("#main-body img") %>%
  html_attr("class")

# Return the dataframe with names, URLs, and images
return(data.frame(
  Name = cheese_names,
  URL = cheese_data,
  Image = cheese_images,
  stringsAsFactors = FALSE
))
}

```

```

# Apply the scrape_page function to all pages of cheese.com
all_cheese_data <- lapply(1:103, scrape_page)

```

This combination of functions shows another example of when we had to iterate over a function in our lab assignment. The function written scrapes one singular page of the cheese.com website. Unfortunately the task was to scrape for all 103 pages of the website and there was not an fairly easy solution for me at first to move through the pages within the function. My idea was then to iterate the scraping of one page for all 103 pages. This was done using the `lapply()` function over the numbers of all numbered pages since the function uses the page number to scrape the desired page. The `lapply()` function uses a vector of all the page numbers and then the function `scrape_page()` that was made for iteration.

[CDAI-6] Generative Art

- I can apply a variety of generative art functions to make a visually pleasing piece.
- I can explain why particular changes to the code result in particular differences in the visualization.

Level: 3

Justification:

```
hourglass_x <- function(angle) {  
  # Formula I played with until it looked right.  
  x <- sin(angle) * abs(cos(angle))^1.2  
  return(x)  
}  
  
# Function to calculate the y-coordinate for an hourglass shape.  
hourglass_y <- function(angle) {  
  y <- cos(angle)  
  return(y)  
}  
  
# Creates a tibble representing the base hourglass shape.  
hourglass_shape <- tibble(  
  angle = seq(0, 2 * pi, length.out = 50),  
  x = hourglass_x(angle),  
  y = hourglass_y(angle)  
)  
  
# Generates data for a single "Perlin noise" hourglass.  
perlin_hourglass2 <- function(n = 100,  
                               freq_init = 0.3,  
                               octaves = 2,  
                               r_min = 0.5,  
                               r_max = 1,  
                               w_min = 0,  
                               w_max = 4,  
                               rot = 0,  
                               x_shift = 0,  
                               y_shift = 0,  
                               id = NA,  
                               seed = NULL) {  
  
  if(!is.null(seed)) set.seed(seed)  
  tibble(  
    angle = seq(0, 2*pi, length.out = n),
```

```

radius = fracture(
    x = cos(angle),
    y = sin(angle),
    freq_init = freq_init,
    noise = gen_perlin,
    fractal = fbm,
    octaves = octaves
) |>
  normalize_radius(r_min, r_max),

# Calculates final x and y coordinates based on radius, hourglass shape, and shifts.
x = radius * hourglass_x(angle) + x_shift,
y = radius * hourglass_y(angle) + y_shift,

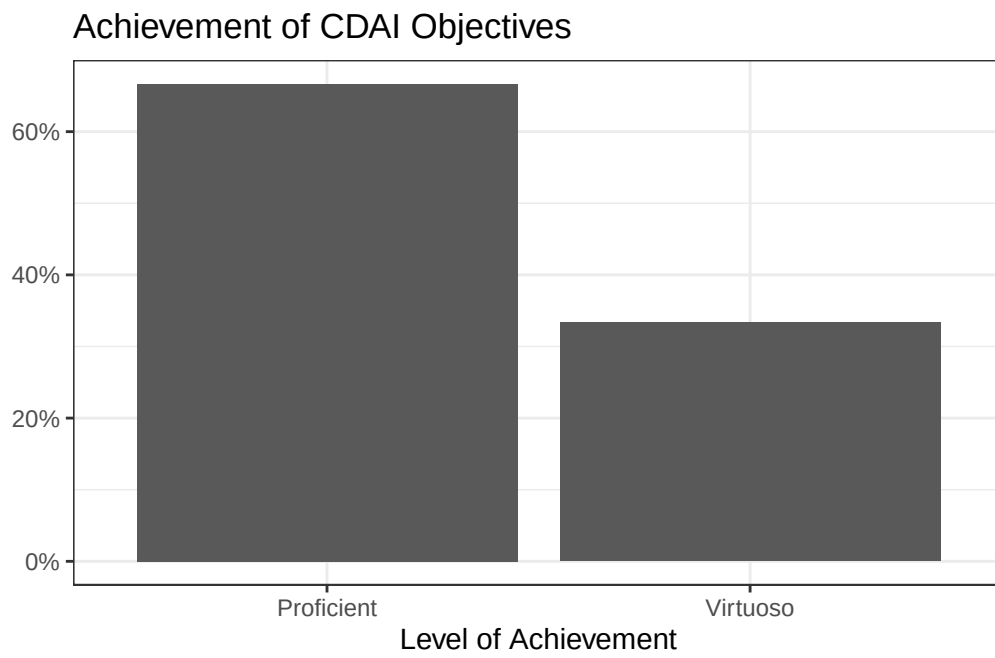
width = fracture(
    x = cos(angle + rot),
    y = sin(angle + rot),
    freq_init = freq_init,
    noise = gen_perlin,
    fractal = fbm,
    octaves = octaves
) |>
  normalize(to = c(w_min, w_max)),

  id = id
)
}

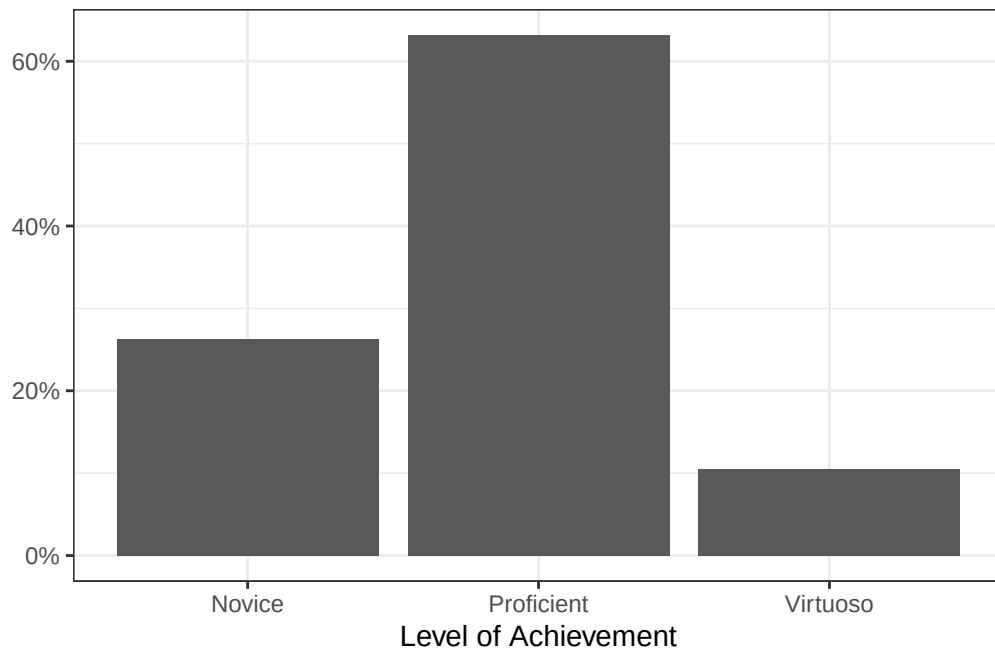
```

So this function shows the main changes I made to apply multiple functions creating the generative art. I created an animated hourglass shape based on a specified palette of colors. The palette of colors corresponds to the number of hourglasses created and the color of each hourglass. The two shape functions at the beginning make the shape of the hourglass and the `perlin_hourglass2()` function creates the single hourglass. There is another function I have that connects all three of these that accepts a palette as one of the input as well as the number of hourglasses to create. All of these functions when combined create a beautiful colored and animated hourglass shape with random noise.

CDAI Summary



Overall Summary



Grade

Based on the summary plot above, I believe I have earned a B- in STAT 541.

Justification

I still feel that I have not achieved virtuoso status in any of the earlier contents from the course. Last time I wanted to get better at commenting code and being more cohesive in clean coding. I believe over the back end of the course I have improved a lot in terms of being reproducible with my code and commenting more. My code has also become a lot cleaner and I feel more effort has been put into my last several labs that my earlier labs were missing. I believe I started to spend more attention to detail since the labs were on new material which was stuff I was interested in learning about. I thoroughly enjoyed learning about APIs and web scraping as this was a skill I had not yet learned before. This helped me put more effort into my work and I believe I thoroughly improved my work towards the end of the quarter. For this reason I believe I earned a B-, since the earlier labs did not hold up to a B standard, but I believe I finished stronger than how I started. I do think there are revisions I could do to the earlier labs to improve my grad to a B+ or A-, but I have not had time to get to the revisions I wanted to for the earlier labs.